

1. Where would you rate yourself on (LLM, Deep Learning, AI, ML).  
A, B, C [A = can code independently; B = can code under supervision; C = have little or no understanding]

Ans:

LLM -> B

Deep Learning -> B

AI-> B

ML->B

2. What are the key architectural components to create a chatbot based on LLM? Please explain the approach on a high-level

Ans:

The LLM chatbot architecture involves different architectural elements working together to allow the chatbot to understand user requests and respond to them.

I think, on a high-level perspective, the chatbot architecture involves the following elements:

**user interface** that allows users to interact with the chatbot, **backend** or application server responsible for request processing, **LLM** that processes the input data and generates responses.

The chatbot architecture may also include conversation memory to maintain interaction history and a **knowledge base** or database to provide additional information. If it is like a RAG chatbot like Perplexity, the chatbot architecture will also involve **retrieval system** to search for relevant data in the knowledge base and return the data to LLM along the user query. Then the response generated by LLM is processed in the backend and provided to the user via the interface.

3. Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why?

Ans:

Vector database is a kind of database designed for storing and searching vectors, which are mathematical representations of data such as text, image or document. Suppose there is a need to develop a college chatbot that responds to the queries of the student based on study material. In that case, I could use vectors for representing the study material and store it in the vector database. The query asked by the user is also transformed into a vector, and then the database searches for those vectors that are close to the query. It makes it possible for the chatbot to locate the relevant data. For this particular problem, I would go for ChromaDB because of its simplicity and beginner-friendliness. I have also worked with FAISS in a LangChain RAG Project.